Project Deep Dive — Jobsy (Complete file-by-file guide)

Purpose

This document is a single-file, comprehensive deep-dive into the Jobsy project in this workspace. It describes the purpose and important implementation details for every key file in the repository (backend and frontend), explains the architecture and data flows, highlights design choices, and includes likely exam-style questions and suggested answers so you can confidently present and defend your project.

Location of this file: `Website/PROJECT_DEEP_DIVE.md`

How to use this file

- Read the high-level architecture first (below) to understand components and responsibilities.
- Use the file lists to drill into specific files; the description explains purpose, important functions, and things you should be ready to discuss.
- Read the "Exam prep" section at the end for common questions and talking points.

High-level architecture

- Frontend: React single-page app in `jobsy-frontend/` (Create React App). Pages under `src/pages`, components under `src/components`, API layer in `src/api.js`.
- Backend: Java + Javalin (micro web framework) in `jobsy-backend/` using simple file-based persistence managed by `FileStorageManager`. Service and DAO layers separate business logic from persistence.
- Data: JSON files used as persistence (`jobs.json`, `users.json`, `applications.json`) kept in the backend root or `data/` directory.
- Auth: Prototype-level. Frontend stores user object in `localStorage` and sends `X-User-Id` header on requests. Backend authorizes requests based on that header (this is a convenience for prototype; not production-grade).

Root / repository files

- `Keys.txt` — Appears at repository root. Possible leaked secret file. If it contains credentials, remove from VCS and add to `.gitignore`.
- `README.md` — Project README (high-level). Useful to run basic instructions.
- `postcss.config.js`, `tailwind.config.js` — Frontend CSS / Tailwind configuration files used by the React app.

Backend — `jobsy-backend/`

This is a Java Maven project using Javalin. The code organizes features into packages: `config`, `dao`, `models`, `services`. The main entrypoint is `JavalinApp.java`.

- `mvnw`, `mvnw.cmd`, `pom.xml` — Maven wrapper and POM for build automation and dependency management. `pom.xml` includes dependencies for Javalin, Jackson, bcrypt, etc.

Key files and what they do

- `src/main/java/com/jobsy/JavalinApp.java`

    - Main application class. Creates DAOs and Services, seeds data via `DataSeeder`, configures Javalin (CORS, JSON mapper), and registers HTTP routes.
    - Routes implemented include:
        - `POST /api/users/signup` — creates a Student or Employer from `SignupRequest` and persists via `UserService`.
        - `POST /api/users/login` — authenticates via `UserService.login`.
        - `GET /api/users/{id}` — returns a user profile; server-side authorization added to allow only the user themself or an employer who has applications from that student (uses `X-User-Id` header).
        - CRUD job endpoints: `GET /jobs`, `GET /jobs/{id}`, `POST /jobs`, `PUT /jobs/{id}`, `DELETE /jobs/{id}`, `GET /jobs/search`, `GET /jobs/employer/{employerId}`.
        - Application endpoints: `GET /applications`, `GET /applications/{id}`, `POST /applications`, `GET /applications/student/{studentId}`, `GET /applications/job/{jobId}` (protected: only job owner can access), `PUT /applications/{id}/accept`, `PUT /applications/{id}/reject`, `DELETE /applications/{id}`.
    - JSON mapping uses Jackson with `JavaTimeModule` and disabled FAIL_ON_UNKNOWN_PROPERTIES for flexibility.
- `src/main/java/com/jobsy/config/DataSeeder.java`
    - Seeds initial data (users and jobs) when application starts. Useful for local development and demos.
- `src/main/java/com/jobsy/models/*` — Data model classes
    - `User.java` (base class) — common fields for Student and Employer. Note: file persistence requires some manual polymorphic handling.
    - `Student.java` — extends `User`. Fields: `skills`, `education`, `major`, `gpa`, etc.
    - `Employer.java` — extends `User`. Fields: `companyName`, `companyDescription`, `industry`, `location`, `website`.
    - `Job.java` — fields include `id`, `title`, `description`, `datePosted`, `salary`, `companyName`, `jobType`, `isOpen`, `employer`, `postedBy`.
    - `Application.java` — fields: `id`, `student` (Student reference), `job` (Job reference), `status`, `dateApplied`.
    - `SignupRequest.java`, `LoginRequest.java` — DTOs for signup/login API payloads.
    - `ErrorResponse.java` — general error response wrapper.
- `src/main/java/com/jobsy/dao/*` — Data Access Objects, file persistence implementation
    - `FileStorageManager.java` — utility to load/save JSON arrays to/from files. Central to file persistence.
    - `UserDAO.java` — manages users; `findAll()` reads `users.json` and manually maps `userType` field to `Student` or `Employer` (Jackson tree-to-value). `save()`, `findById()`, `findByEmail()`, `delete()` are implemented.
    - `JobDAO.java` — manages `jobs.json`. `save()` inserts/updates with incremental numeric IDs; `loadAllAndHydrate()` populates `Employer` references by reading users.
    - `ApplicationDAO.java` — manages `applications.json`. On save, checks for duplicate applications by same student for same job; `loadAllAndHydrate()` hydrates student and job references.

Important behavior notes (backend DAOs)

- ID generation: incrementally set to max existing id + 1. Not safe for concurrent writes but works for prototype.
- Hydration: because file storage may store only IDs or shallow objects, DAOs re-load related entities (users/jobs) and replace references with full objects for API responses.
- Concurrency and data corruption: file-based persistence is simple but not safe for concurrent production use. Explainable trade-off for a student project.
- `src/main/java/com/jobsy/services/*` — Business logic
  - `UserService.java` — contains `save`, `login`, `findById` wrappers around `UserDAO` and any higher-level checks.
  - `JobService.java` — wrappers around `JobDAO` to fetch jobs and business-level actions.
  - `ApplicationService.java` — handles application submission and business rules (default status, accept/reject flow) and calls `ApplicationDAO`.

Backend design choices to discuss

- Separation of concerns: Controllers (Javalin routes) -> Services -> DAOs -> File storage.
- Why DTOs? `SignupRequest` and `LoginRequest` isolate API payloads from internal models and allow validation/transformations.
- Error handling: routes catch exceptions and return `ErrorResponse` or appropriate HTTP status codes.

Frontend — `jobsy-frontend/`

This is a React app (Create React App) using Tailwind and Framer Motion for visuals. Important parts are `src/api.js`, `src/pages/*`, and `src/components/*`.

Top-level files

- `package.json` — dependencies and scripts. Scripts include `start`, `build`, `test` etc.
- `postcss.config.js`, `tailwind.config.js` — Tailwind setup.

Key frontend files (by folder)

- `src/api.js`
  - Centralized API wrapper using `axios` with `baseURL: http://localhost:8080`.
  - Exposes functions for: `signupUser`, `loginUser`, `getJobs`, `getJobById`, `createJob`, `updateJob`, `deleteJob`, `searchJobs`.
  - Application endpoints: `applyJob`, `getApplicationsByStudent`, `getApplicationsByJob`, `getAllApplications`, `acceptApplication`, `rejectApplication`.
  - User endpoint: `getProfile`.
  - A request interceptor was added to attach `X-User-Id` header (from `localStorage.user`) so backend can perform simple auth checks.
- `src/pages/*` (major pages)
  - `Home.jsx` — Landing page; may show featured items or calls-to-action.
  - `Jobs.jsx` — Jobs listing page. Uses `getJobs()` to show job cards, search, and link to job details.
  - `JobDetails.jsx` — Single-job page. Responsibilities:
    - Load job via `getJobById(id)`.
    - Allow a logged-in student to apply for the job (calls `applyJob`).

        - If the logged-in user is the employer who posted the job, it loads applicants via `getApplicationsByJob(job.id)` and displays them.
        - Shows Accept / Reject buttons for pending applications; calls `acceptApplication` / `rejectApplication` and updates the local state.
    - `PostJob.jsx` — Form to create a new job (calls `createJob`). Only available to employers.
    - `Login.jsx` / `Signup.jsx` — Authentication forms. On successful login, saves the returned user object into `localStorage` (prototype auth) and userType into localStorage.
    - `Profile.jsx` — Shows user profile. It supports two modes:
        - Own profile: reads `localStorage.user` and refreshes via `getProfile(userId)`.
        - Viewing someone else's profile: route `/profile/:id` — fetches that user's profile from the server and shows it. When viewing another profile the Logout button is hidden.

- `src/components/*`
    - `Navbar.jsx` — top navigation, responsive. Exposes login/logout links based on `localStorage.user`.
    - `Layout.jsx` — common layout wrapper (if present) used by pages.
    - Small UI components: `FluidScroll.jsx`, `ScrollStack.jsx`, `TextPressure.jsx` and background components under `Backgrounds/` for styling.

Frontend behavior and data flow

- `localStorage`: stores the serialized `user` object after login/signup. Components read this to determine available actions (apply, post job) and `X-User-Id` is attached automatically to API requests.
- API calls: `api.js` centralizes HTTP calls so pages/components can be simple and focused on UI.
- Routing: `App.js` defines routes, and `Profile.jsx` supports viewing other profiles with `useParams()`.

Data files and generated artifacts

- `jobsy-backend/jobs.json` — persisted job list.
- `jobsy-backend/users.json` — persisted users list; `userType` field used to distinguish Student/Employer.
- `jobsy-backend/applications.json` — persisted applications.
- `jobsy-backend/target/` — Maven build output (do not track in VCS).
- `jobsy-frontend/build/` — production build artifacts.

Security & production considerations (what examiners will ask)

- Authentication: current approach trusts `localStorage` and sends `X-User-Id` header. This is not secure for production. Use JWTs (signing) or server sessions and verify tokens on every request.
- Authorization: server-side checks were added for important endpoints, but you should implement verified identity via tokens.
- Persistence: file-based storage is easy for demos but has drawbacks:
    - No concurrency control (race conditions possible on write).
    - No transactions, limited querying, and larger data becomes slow.
    - Suggestion: migrate to SQLite or PostgreSQL and use JDBC or an ORM (e.g., JDBI, Hibernate) for production.
- Input validation: ensure server validates and sanitizes inputs. At present some inputs are read directly into models.
- Secret management: remove `Keys.txt` from repo and rely on environment variables or a secret manager.

Common exam questions and suggested answers

Q: Why did you use file-based persistence instead of a database?

- A: For simplicity and speed of development in a small-scale project. It removes the need for DB setup in demos. I am aware of the drawbacks (concurrency, durability, querying) and have a migration plan: create a single `DataSource` abstraction and implement a `JdbcUserDAO` / `JdbcJobDAO` to replace file-backed DAOs.

Q: How does the application prevent an employer from seeing profiles of students who did not apply?

- A: The backend `GET /api/users/{id}` endpoint checks the caller id (from `X-User-Id`) and allows access if:
  - caller id equals requested id (user viewing their own profile), OR
  - caller is an employer who has an application record where the student id matches the requested id and the application references a job owned by the caller. This logic ensures employers only view applicant profiles.

Q: How does accept/reject work under the hood?

- A: `PUT /applications/{id}/accept` and `PUT /applications/{id}/reject` set the `status` field on the `Application` object (e.g., `ACCEPTED`, `REJECTED`) and call `applicationDAO.save(application)` which persists the updated application back to `applications.json`.

Q: How do you generate IDs? Any potential problem?

- A: For each entity, the DAOs compute `max(existingIds) + 1` and assign it. This is simple but has race conditions if two saves occur simultaneously — both may compute the same max ID and write conflicting data. Production would require a database with atomic ID generation or a locking mechanism.

Q: Why separate DAOs and Services?

- A: DAOs manage persistence (read/write), while Services encapsulate business rules—this separation improves maintainability, testability, and allows swapping persistence implementation easily.

Q: How does frontend know which user is logged in?

- A: After successful login/signup the frontend stores the returned user object in `localStorage` (key: `user`). Components read this to determine available actions (apply, post job) and `X-User-Id` is attached automatically to API requests.

Q: Could a malicious client spoof `X-User-Id` to access another user's data?

- A: Yes — this approach trusts the client. That's why in production you must use signed tokens (JWT) or server-managed sessions. The current `X-User-Id` header is a prototype convenience.

Potential follow-up improvements you should be ready to discuss

1. Replace `X-User-Id` with JWT authentication:
   - Implement login to return a signed JWT.
   - Add middleware to validate JWT on each request and expose the authenticated user id.
   - Remove reliance on `localStorage` for identity; still store token but validate on server.
2. Replace file persistence with a relational DB:
   - Add `Jdbc` implementations of DAOs.

    - Use transactions for application acceptance (e.g., when accepting you might want to close job or send a message).

3. Add tests & CI:
    - Unit tests for DAOs/services.
    - Integration tests to exercise the Javalin endpoints (use an in-memory DB or temporary JSON files).
    - Add GitHub Actions workflow to run build/test on push.

4. Improve concurrent safety (interim step without DB):
    - Implement file-level locking when writing JSON files (synchronized blocks or OS-level lock), or use append-only log and compaction.

File-by-file reference (concise map)

Below is a concise list of important files and a one-line summary for each (useful as flash-cards):

Root

- `Keys.txt` — (sensitive?) keys/secrets. Remove.
- `postcss.config.js` — CSS pipeline config (Tailwind)
- `tailwind.config.js` — Tailwind theme config

Backend (jobsy-backend)

- `pom.xml`, `mvnw`, `mvnw.cmd` — Maven build configuration/wrapper.
- `README.md` — backend README (if present).
- `src/main/java/com/jobsy/JavalinApp.java` — Main app and route registration.
- `src/main/java/com/jobsy/config/DataSeeder.java` — Seeds initial data for quick demos.

Models

- `src/main/java/com/jobsy/models/User.java` — Base user fields; polymorphism handled manually in DAO.
- `src/main/java/com/jobsy/models/Student.java` — Student profile fields.
- `src/main/java/com/jobsy/models/Employer.java` — Employer profile fields.
- `src/main/java/com/jobsy/models/Job.java` — Job posting model.
- `src/main/java/com/jobsy/models/Application.java` — Application model linking student & job.
- `src/main/java/com/jobsy/models/SignupRequest.java` — Signup request DTO.
- `src/main/java/com/jobsy/models/LoginRequest.java` — Login request DTO.
- `src/main/java/com/jobsy/models/ErrorResponse.java` — Error payload.

DAO layer

- `src/main/java/com/jobsy/dao/FileStorageManager.java` — JSON read/write helper.
- `src/main/java/com/jobsy/dao/UserDAO.java` — CRUD for users (file-backed), handles polymorphism.
- `src/main/java/com/jobsy/dao/JobDAO.java` — CRUD for jobs; hydrates employer data.
- `src/main/java/com/jobsy/dao/ApplicationDAO.java` — CRUD for applications; prevents duplicates; hydrates student/job.

Services

- `src/main/java/com/jobsy/services/UserService.java` — Higher-level user operations and login.
- `src/main/java/com/jobsy/services/JobService.java` — Job-related business logic.

- `src/main/java/com/jobsy/services/ApplicationService.java` — Application submission and status changes.

Frontend (jobsy-frontend)

- `package.json` — dependencies and scripts.
- `public/index.html` — SPA mount point.
- `src/index.js` — React app bootstrapping.
- `src/api.js` — Centralized REST API wrapper with `axios` and `X-User-Id` header injector.

Pages

- `src/pages/Home.jsx` — Landing page.
- `src/pages/Jobs.jsx` — Jobs list and search.
- `src/pages/JobDetails.jsx` — Job detail page, apply + applicant management (Accept/Reject).
- `src/pages/PostJob.jsx` — Employer job posting form.
- `src/pages/Login.jsx` — Login form.
- `src/pages/Signup.jsx` — Signup form (student and employer).
- `src/pages/Profile.jsx` — Shows user profile; supports `/profile` and `/profile/:id` views.

Components

- `src/components/Navbar.jsx` — Top navigation and responsive behavior.
- `src/components/Layout.jsx` — Page frame (if present).
- `src/components/Backgrounds/*` — Visual background components (Silk, FluidGlass, etc.) used for UI polish.

Build artifacts and others

- `jobsy-backend/target/` — Maven build outputs (jar). Ignore for VCS.
- `jobsy-frontend/node_modules/` — dependencies installed by npm. Ignore for VCS.
- `jobsy-frontend/build/` — production build output.

Practical study checklist (what to memorize)

1. Be able to draw the system architecture: browser → React app → Javalin server → file storage.
2. Explain the happy paths: signup/login → create job → student apply → employer view applicants → accept/reject.
3. Explain each layer's responsibility: controllers/routes, services, DAOs, file storage.
4. Be ready to justify trade-offs (file storage simplicity vs DB robustness).
5. Understand the security gaps and clear plan to fix them (JWT, DB, input validation).

Sample exam answers (short, ready-to-repeat)

- "The application separates responsibilities into routes (Javalin), services (business rules), and DAOs (persistence). This separation makes it easier to test and to swap out persistence later (for example switching to JDBC)."
- "We used JSON files for persistence because it's simple for a prototype and avoids requiring a DB server, but it is not suitable for concurrent production use."
- "We plan to replace `X-User-Id` with JWT-based authentication: on login the server will return a signed token, the client will store it in memory or a secure cookie, and the server will validate the token on every request and derive the authenticated user id from it."

Appendix — Quick grep map (commands you can run locally)

To find code quickly:

```
# list backend source files
ls -Recurse -File Website\jobsy-backend\src\main\java\com\jobsy | select FullName

# find occurrences of `applications` or `getApplicationsByJob`
Select-String -Path Website\jobsy-backend\src\main\java\**\*.java -Pattern
"getApplicationsByJob" -List

# search frontend for API usages
Select-String -Path Website\jobsy-frontend\src\**\*.js* -Pattern
"getApplicationsByJob|applyJob|acceptApplication" -List
```

Closing notes

This single-file guide is intended to be your study companion when preparing to present or defend your project. If you'd like, I can:

- Generate a one-page cheat sheet (two-column A4 PDF) summarizing endpoints, models, and common commands.
- Produce sample interview questions and a mock oral exam script and role-play it with you.

If you want me to expand any section (for example produce a line-by-line walkthrough of `JavalinApp.java` routes or the exact JSON structure of persisted files), tell me which file and I will append a fully annotated version to this same document.

Good luck — let's make your defense unbeatable.